

RL Homework 2 problem 4

110612025 魏于翔

March 2026

1 (a)

1.1 Implementation

In this problem, we implement DDPG algorithm on Pendulum-v1 problem. We design Actor, Critic DDPG and train loop.

Actor: The NN structure is (**num input** $\rightarrow 256 \rightarrow 256 \rightarrow$ **num outputs**), using ReLU activation for the hidden layers, and Tanh for the output layer, because the action space is typically bounded between -2 and 2 for Pendulum-v1.



```
class Actor(nn.Module):
    def __init__(self, hidden_size, num_inputs, action_space):
        super(Actor, self).__init__()
        self.action_space = action_space
        num_outputs = action_space.shape[0]

        self.hidden_size1 = 256
        self.hidden_size2 = 256

        self.max_action = 2

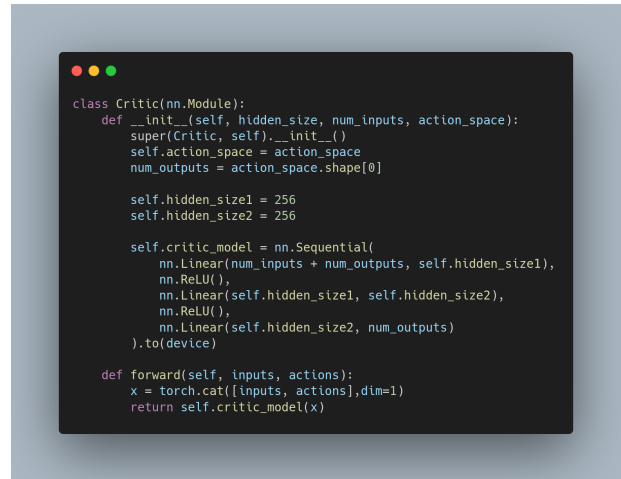
        self.actor_model = nn.Sequential(
            nn.Linear(num_inputs, self.hidden_size1),
            nn.ReLU(),
            nn.Linear(self.hidden_size1, self.hidden_size2),
            nn.ReLU(),
            nn.Linear(self.hidden_size2, num_outputs),
            nn.Tanh()
        ).to(device)

    def forward(self, inputs):
        return self.max_action * self.actor_model(inputs)
```

Figure 1: Actor class

Critic: The NN structure is (**num input** $\rightarrow 256 \rightarrow 256 \rightarrow$ **num outputs**), using ReLU activation for the hidden layers, no activation for the output layer,

since the output is a numeric Q-value that can be unbounded.



```
class Critic(nn.Module):
    def __init__(self, hidden_size, num_inputs, action_space):
        super(Critic, self).__init__()
        self.action_space = action_space
        num_outputs = action_space.shape[0]

        self.hidden_size1 = 256
        self.hidden_size2 = 256

        self.critic_model = nn.Sequential(
            nn.Linear(num_inputs + num_outputs, self.hidden_size1),
            nn.ReLU(),
            nn.Linear(self.hidden_size1, self.hidden_size2),
            nn.ReLU(),
            nn.Linear(self.hidden_size2, num_outputs)
        ).to(device)

    def forward(self, inputs, actions):
        x = torch.cat([inputs, actions], dim=1)
        return self.critic_model(x)
```

Figure 2: Critic class

DDPG: For the DDPG class, i implement two main functions, select action and update parameters, for select action function, we add noise on action for exploration, add clipping the action in the range -2 to 2, since it's this problem's action range.

As for the update parameters function, the main purpose of this function is to update the Critic and Actor neural network, i first update the critic network by calculating the target Q value and current Q value, and calculate the mean square error of these two values for the value loss, using it to update the parameters of critic network, as for actor network, since we want to maximize the Q value, i take negative mean of the critic's Q value as the loss function to minimize the loss, and use that loss function to update

the actor network's parameters.

```
##### YOUR CODE HERE (3~5 lines) #####
# Add noise to your action for exploration
# Clipping might be needed

if action_noise is not None:
    noise = action_noise.noise()
    noise = torch.FloatTensor(noise).to(device)
    mu += noise
    clip_mu = torch.clamp(mu, min=-2.0, max=2.0)

    return clip_mu.cpu().numpy()

return mu.cpu().numpy()
```

Figure 3: select action

```
##### YOUR CODE HERE (10~20 lines) #####
# Calculate policy loss and value loss
# Update the actor and the critic

# Part 1. Update critic network
target_Q = reward_batch + (mask_batch * self.gamma * next_state_action_values)
current_Q = self.critic(state_batch, action_batch)
value_loss = F.mse_loss(current_Q, target_Q.detach())

self.critic_optim.zero_grad()
value_loss.backward()
self.critic_optim.step()

# Part 2. Update actor network

policy_loss = -self.critic(state_batch, self.actor(state_batch)).mean()
self.actor_optim.zero_grad()
policy_loss.backward()
self.actor_optim.step()

##### END OF YOUR CODE #####

# Update the target networks here with soft update
soft_update(self.actor_target, self.actor, self.tau)
soft_update(self.critic_target, self.critic, self.tau)

return value_loss.item(), policy_loss.item()
```

Figure 4: Update parameter

1.2 Results

At episode 322, the score (emwa rewards) is at least -130.

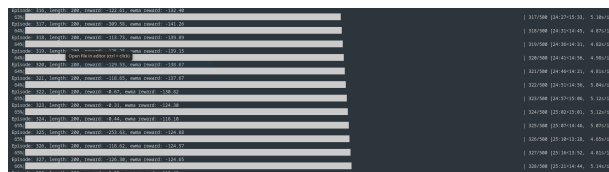


Figure 5: threshold

I use the best-fitted model on test function, the results of the 20 episodes are that basically majority of the episode's reward is higher than -130.

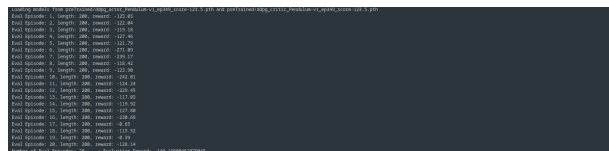
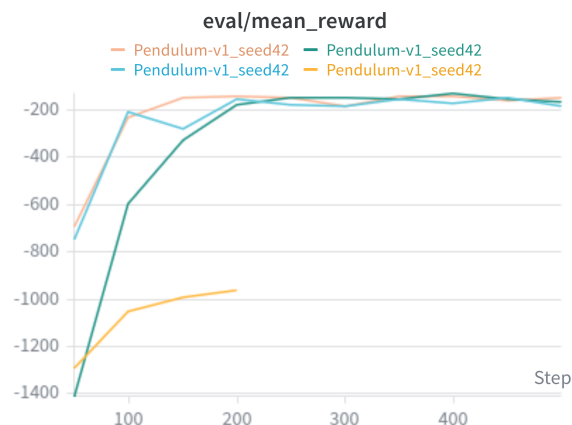
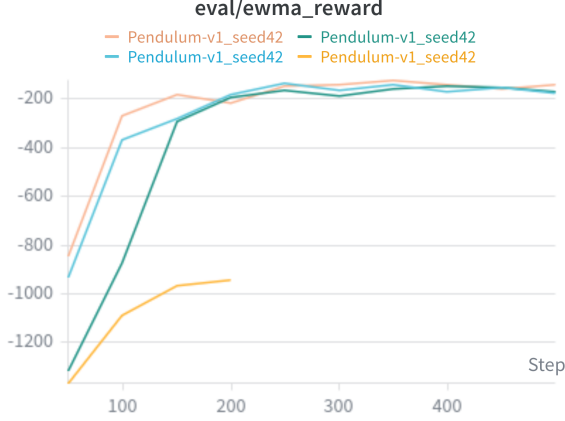


Figure 6: Test

I screen shot the results of the mean reward and emwa reward to check the process of the agent learn.





1.3 Hyperparameters

Hyperparameters's setting take a important part of training, so following are some value of the hypermaters in this training.

- batch size = 256
- $lr_a = 3e - 4$
- $lr_c = 3e - 4$
- noise scale start=0.2
- update per step = 1

2 (b)

In this problem, change the problem from Pendulum-v1 to HalfCheetah-v5, the problem become more complex, so we have to make some adjustment based on the original DDPG algorithm structure to let the model's performance reach our expectation, that is an average evaluation score of a least 5000 within 500k steps.

2.1 Implementation

We change the neural networks structure of both actor and critic networks, the original structure is

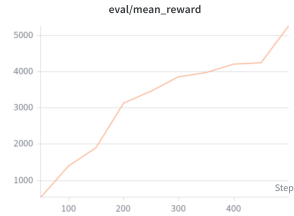
(**num input** $\rightarrow 256 \rightarrow 256 \rightarrow$ **num outputs**), we change it to (**num input** $\rightarrow 400 \rightarrow 300 \rightarrow$ **num outputs**). Let the network can capture the features, since this problem's action and state space have higher dimension. After several times of failure, i observed the value loss curve and found out it sometimes made

2.2 Results

After many times trial and error, i finally get one set of hyperparameters that can reach the mean evaluation score to 5264.



And the following are the mean reward and emwa reward of this training.



2.3 Hyperparameters

Since halfcheetah is more challenging environment, the hyperparameters's value will affect the training results, so i record the values.

- batch size = 512
- $lr_a=2e-4$
- $lr_c=3e-4$
- noise scale start=0.3
- update per step = 2

3 (c)

3.1 Implementation

In algorithm like DDPG, the Q value are often been overestimated, so we implement CDQ (clipping double Q) method to suppress the Q value. In DDPG class, we first build another critic and target critic networks, and in the update parameters function, we calculate the two different critic network's Q values and choose the small to calculate the target Q value, in this method, we can suppress the overestimating, and make training more stable.

```
class DDPG(object):
    def __init__(self, num_inputs, action_space, gamma=0.99, tau=0.001, hidden_size=256, lr_a=3e-4, lr_c=3e-4):
        self.num_inputs = num_inputs
        self.action_space = action_space

        self.actor = Actor(hidden_size, self.num_inputs, self.action_space).to(device)
        self.actor_target = Actor(hidden_size, self.num_inputs, self.action_space).to(device)
        self.actor_perturbed = Actor(hidden_size, self.num_inputs, self.action_space).to(device)
        self.actor_optim = Adam(self.actor.parameters()), lr(lr_a)

        self.critic = Critic(hidden_size, self.num_inputs, self.action_space).to(device)
        self.critic_target = Critic(hidden_size, self.num_inputs, self.action_space).to(device)
        self.critic_optim = Adam(self.critic.parameters()), lr(lr_c)

        # Add another critic for CDQ
        self.critic2 = Critic(hidden_size, self.num_inputs, self.action_space).to(device)
        self.critic_target2 = Critic(hidden_size, self.num_inputs, self.action_space).to(device)
        self.critic_optim2 = Adam(self.critic2.parameters()), lr(lr_c)

        self.gamma = gamma
        self.tau = tau

        hard_update(self.actor_target, self.actor) # Make sure target is with the same weight
        hard_update(self.critic_target, self.critic)
        hard_update(self.critic_target2, self.critic2)
```

```
##### YOUR CODE HERE (10-20 lines) #####
# Calculate policy loss and value loss
# Update the actor and the critic

# Key part of CDQ: Add clipped noise on target action
with torch.no_grad():
    noise = torch.FloatTensor(action_batch.shape).normal_(0, 0.2).to(device) # Clipped noise with std=0.2, same as the paper
    noise = torch.clamp(noise, min=-0.5, max=0.5) # Clamp the noise to the valid range
    next_action_batch = (self.actor_target(next_state_batch)+noise).clamp(-1.0,1.0) # Add noise to the target action and clamp it into the valid range of action space.

# Calculate the two target Q values from the two critic target networks
next_state_action_values_1 = self.critic_target(next_state_batch, next_action_batch)
next_state_action_values_2 = self.critic_target2(next_state_batch, next_action_batch)
# Take the minimum of the two target Q values to mitigate overestimation bias.
next_state_action_values = torch.min(next_state_action_values_1, next_state_action_values_2)

# Compute the target Q value using the reward and the discounted next state action value
target_Q = reward_batch + (mask_batch * self.gamma * next_state_action_values)

# Part 1: Update critic network
current_Q = self.critic(state_batch, action_batch)
value_loss = F.mse_loss(current_Q, target_Q.detach())

# Optimize the critic network
self.critic_optim.zero_grad()
value_loss.backward()
torch.nn.utils.clip_grad_norm(self.critic.parameters(), 1.0)
self.critic_optim.step() # Update the critic network parameters

# Update the second critic network for CDQ
current_Q2 = self.critic2(state_batch, action_batch)
value_loss2 = F.mse_loss(current_Q2, target_Q.detach())
self.critic_optim2.zero_grad()
value_loss2.backward()
torch.nn.utils.clip_grad_norm(self.critic2.parameters(), 1.0)
self.critic_optim2.step()

policy_loss = -self.critic(state_batch, self.actor(state_batch)).mean()

# Optimize the actor network
self.actor_optim.zero_grad()
policy_loss.backward()
self.actor_optim.step()

##### END OF YOUR CODE #####

# Update the target networks here with soft update
soft_update(self.actor_target, self.actor, self.tau)
soft_update(self.critic_target, self.critic, self.tau)
soft_update(self.critic_target2, self.critic2, self.tau)

value_loss = (value_loss + value_loss2) / 2

return value_loss.item(), policy_loss.item()
```

3.2 Results

I use the same hyperparameters in problem (b) to see how well the model will perform when added the CDQ method, and the result was stunning, it increased nearly 2000 score in the evaluation test. This result indicate that if the Q value is been overestimated, how much influence will it make to the model's performance.

```
python3 run.py --env=csd_cheetah --pretrained_critic=csd_cheetah_actor_halfcheetah-v5_4q499_30000756_8.pth and pretrained_critic=csd_cheetah_critic_halfcheetah-v5_4q499_30000756_8.pth

Loading models from pretrained_critic=csd_cheetah_actor_halfcheetah-v5_4q499_30000756_8.pth and pretrained_critic=csd_cheetah_critic_halfcheetah-v5_4q499_30000756_8.pth
Eval Episode: 1, length: 1000, reward: 7833.74
Eval Episode: 2, length: 1000, reward: 6897.28
Eval Episode: 3, length: 1000, reward: 6988.80
Eval Episode: 4, length: 1000, reward: 6977.56
Eval Episode: 5, length: 1000, reward: 6956.49
Eval Episode: 6, length: 1000, reward: 6885.18
Eval Episode: 7, length: 1000, reward: 6955.34
Eval Episode: 8, length: 1000, reward: 6991.90
Eval Episode: 9, length: 1000, reward: 6921.79
Eval Episode: 10, length: 1000, reward: 7117.55
Eval Episode: 11, length: 1000, reward: 7843.38
Eval Episode: 12, length: 1000, reward: 6874.98
Eval Episode: 13, length: 1000, reward: 6962.23
Eval Episode: 14, length: 1000, reward: 7067.99
Eval Episode: 15, length: 1000, reward: 7159.67
Eval Episode: 16, length: 1000, reward: 7112.01
Eval Episode: 17, length: 1000, reward: 6912.45
Eval Episode: 18, length: 1000, reward: 6997.48
Eval Episode: 19, length: 1000, reward: 7047.17
Eval Episode: 20, length: 1000, reward: 7003.84

Number of Eval Episodes: 20, Evaluation Reward: 6956.41647294292
```

Figure 7: evaluation performance

The following are the mean rewards and emwa rewards in the training process.

